

Deep Q-Networks — relatório experimental

Vinicius Tormin e Bruno Calura · Aprendizado por Reforço, UFG · CartPole-v1, 500k passos, 2 seeds por configuração

Fork com o `algorithms/dqn.py` completo: github.com/viniciustormin/t-zero-learning

Protocolo. Baseline = `configs/dqn_cartpole.yml` sem alterações; cada varredura muda um único hiperparâmetro via `--override`, com seeds 1 e 2. As **previsões foram escritas e commitadas antes de qualquer figura existir** (`report/predictions.md`, commit anterior ao da geração das figuras). Nas figuras, a linha grossa é a média entre seeds e a faixa é a envoltória dos dois seeds; cor mais escura = valor maior do hiperparâmetro. Curvas suavizadas por EMA ($\alpha = 0,10$; $\alpha = 0,04$ no TD loss). Não houve conta no W&B disponível (o modo anônimo foi descontinuado pelo serviço, erro 401), então os runs rodaram com `WANDB_MODE=offline` e as figuras foram geradas a partir do espelho local de cada `wandb.log` (`tools/train_csv.py` → `sweep_results/<config>__seed<N>/metrics.jsonl`); os diretórios offline do W&B ficam preservados em `wandb/` e podem ser sincronizados com `wandb sync`.

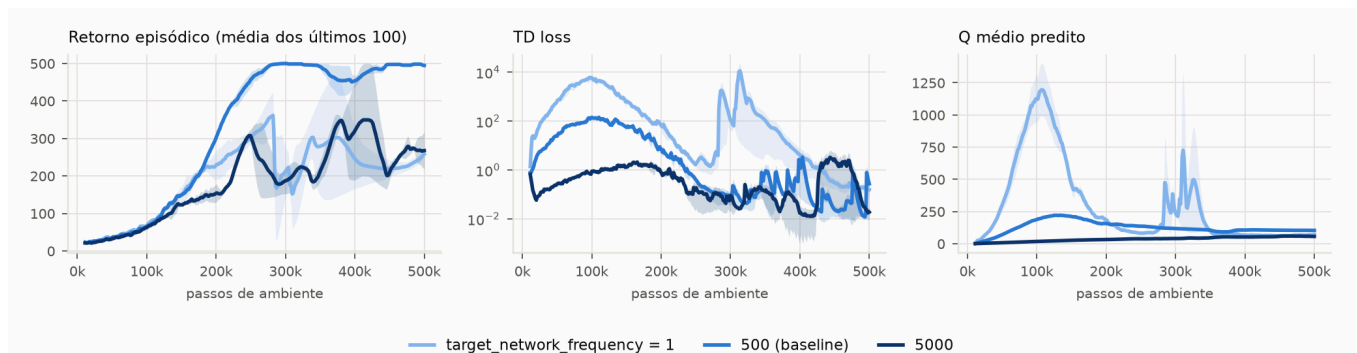
Q1 — Frequência de sincronização da target network

Previsão

Com `tnf=1` e `tau=1.0` a rede-alvo vira uma cópia da rede online a cada passo: o alvo persegue a própria predição e o viés de maximização se realimenta, então esperávamos `q_values` estourar muito acima do teto real (≈ 100 para $\gamma=0,99$ e recompensa 1 por passo) e o retorno degradar, com `td_loss` **baixo** porque o erro seria medido contra um alvo que se move junto. Com `tnf=5000` (100 sincronizações em todo o run) esperávamos `td_loss` em dentes de serra e aprendizado mais lento. Baseline (500) seria o melhor dos três.

Varredura e gráficos

`dqn.target_network_frequency` ∈ {1, 500 (baseline), 5000}, seeds 1 e 2.



Explicação

O papel da target network é tornar o alvo **semi-estacionário**: ela congela $\max_a Q(s', a)$ por N passos, de modo que a regressão do passo de gradiente tenha um alvo fixo em vez de um que se desloca junto com os pesos que estão sendo atualizados. Os dois extremos quebram isso de formas opostas, e é o painel de `q_values` que separa uma da outra. Com `tnf=1` o alvo é recalculado com os pesos recém-atualizados: o erro de superestimação do operador $\max$ é reinjetado no alvo a cada passo e se amplifica — `q_values` sobe até ≈ 1321 , treze vezes o retorno descontado máximo possível (≈ 100), e o `td_loss` mediano chega a **28,6** (689 na primeira janela de 100k), ou seja, a regressão nunca alcança o alvo porque o alvo foge. Com `tnf=5000` acontece o contrário: `q_values` estaciona em ≈ 62 , **abaixo** do valor verdadeiro, porque cada sincronização propaga apenas um passo de bootstrap e 100 sincronizações não bastam para o valor atravessar um horizonte de ≈ 500 passos. O retorno reflete isso: baseline 497 nos últimos 50k passos, contra 231 (`tnf=1`) e 259 (`tnf=5000`).

E aqui está a resposta direta a “por que o retorno pode degradar enquanto o `td_loss` continua parecendo bem”: **o pior `td_loss` do nosso sweep não é o da pior curva de retorno**. `tnf=5000` tem o `td_loss` mediano mais baixo de todas as configurações da Q1 (**0,11**, contra 0,35 do baseline) e ainda assim um retorno quase 250 pontos pior. Com o alvo congelado por 5000 passos, a rede tem 500 atualizações de gradiente para ajustar um alvo fixo — e consegue, quase perfeitamente. O `td_loss` mede **o quanto $Q(s,a)$ se aproximou do alvo**, não **se o alvo estava certo**; um alvo obsoleto e bem ajustado produz erro pequeno e política ruim. Ler as duas curvas juntas é o que desfaz a ambiguidade. **Previsão: parcialmente confirmada.** Acertamos a explosão de `q_values` em `tnf=1`, a lentidão de `tnf=5000` e a vitória do baseline; erramos de lado na atribuição do `td_loss` baixo — supusemos que seria `tnf=1` (que na verdade tem o maior), quando o caso “loss ótimo, política ruim” é o do alvo quase congelado.

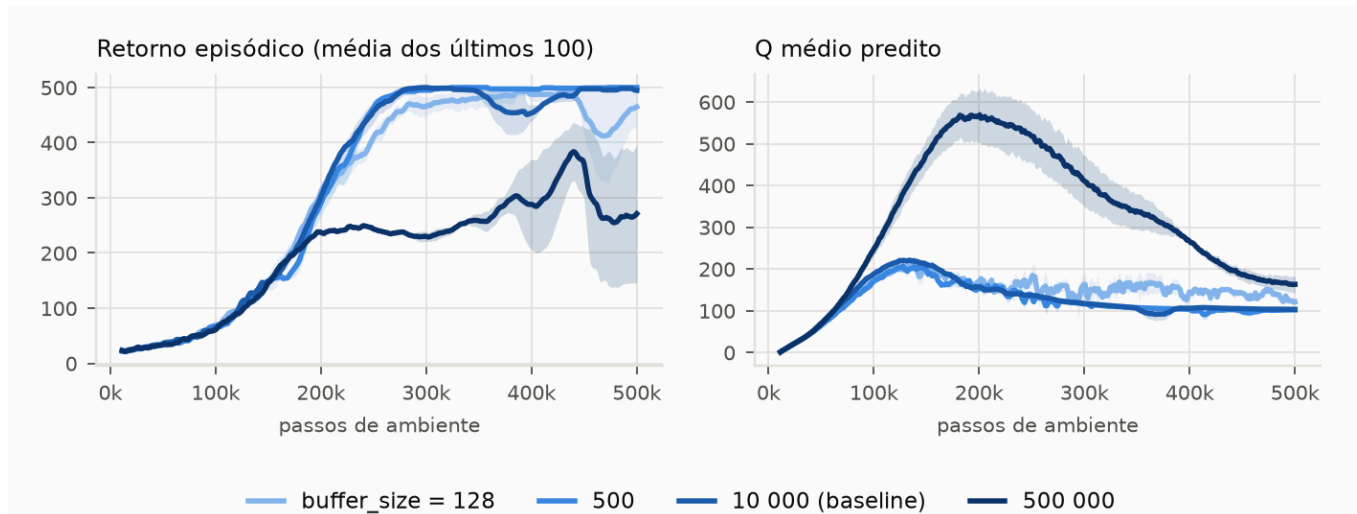
Q2 — Tamanho do replay buffer

Previsão

Com `buffer=500` o minibatch de 128 sai de uma janela de $\approx 1-2$ episódios: amostras fortemente correlacionadas e quase on-policy, e transições antigas despejadas antes de a rede consolidá-las — esperávamos curva instável (sobe e desaba) e `q_values` ruidoso. Com `buffer=500000` nada é despejado em 500k passos: esperávamos a curva **mais estável** das quatro, talvez mais lenta no início.

Varredura e gráficos

`dqn.buffer_size` $\in \{128, 500, 10\,000 \text{ (baseline)}, 500\,000\}$, seeds 1 e 2. O ponto 128 foi acrescentado depois de ver que 500 não degradava: com `batch_size=128`, um buffer de 128 faz o **minibatch ser o buffer inteiro** — uma fatia consecutiva de trajetória — que é o regime em que o problema da correlação realmente aparece.



Explicação

Os dois problemas que um buffer pequeno causa são distintos e aparecem em painéis distintos. O primeiro é de **correlação**: as 128 transições de um minibatch vêm todas da mesma janela curta de trajetória, quase consecutivas, então o gradiente do batch é essencialmente o gradiente de um punhado de estados vizinhos — a decorrelação que é a razão de existir do replay desaparece e cada atualização puxa a rede na direção do trecho mais recente. O segundo é de **cobertura no tempo**: o buffer é a única memória que a rede tem, e o que sai dele nunca mais é revisto; regiões do espaço de estados que o agente dominou deixam de aparecer nos batches, a rede deixa de ser treinada nelas e as esquece (esquecimento catastrófico). O primeiro estraga **cada** atualização; o segundo estraga a **retenção** entre atualizações.

Nos nossos dados, os dois só se manifestam em `buffer=128`, e na assinatura esperada: a curva **atinge** 500 por volta de 250k passos e depois não a sustenta, recaindo nas janelas de 380–420k e 460–500k, com os dois seeds se separando (retorno médio nos últimos 50k: 372 e 497; avaliação greedy final 344 e 500) e `q_values` inflado a ≈ 135 contra os ≈ 100 verdadeiros. Subir, cair e subir de novo é exatamente o que esquecimento produz — não é ruído, é a rede perdendo regiões que já sabia. Já `buffer=500` é indistinguível do baseline (500 vs. 497), e aqui a previsão falhou: 500 transições ainda são vários episódios no começo do treino, e o CartPole tem um espaço de estados de 4 dimensões, suave o bastante para que uma MLP pequena tolere batches correlacionados. **O fracasso maior deste sweep foi o buffer que nunca esquece.** Com `buffer=500000` nada é despejado, então metade do buffer permanece sendo dados da política quase aleatória do início; a distribuição de estados sobre a qual Q é ajustado nunca converge para a distribuição da política que está sendo melhorada. O painel de `q_values` mostra o efeito com clareza: um monte até ≈ 603 — seis vezes o valor verdadeiro — que decai lentamente conforme dados melhores diluem os antigos, e um `td_loss` mediano de 134, quase 400× o do baseline. O retorno nunca chega a 450 em nenhum dos dois seeds. **Previsão: invertida.** Prevíamos que o extremo pequeno seria o instável e o extremo grande o estável; observamos o oposto, e a moral é que “quantos dados cabem” importa menos do que “de qual política são os dados que ainda estão lá”.

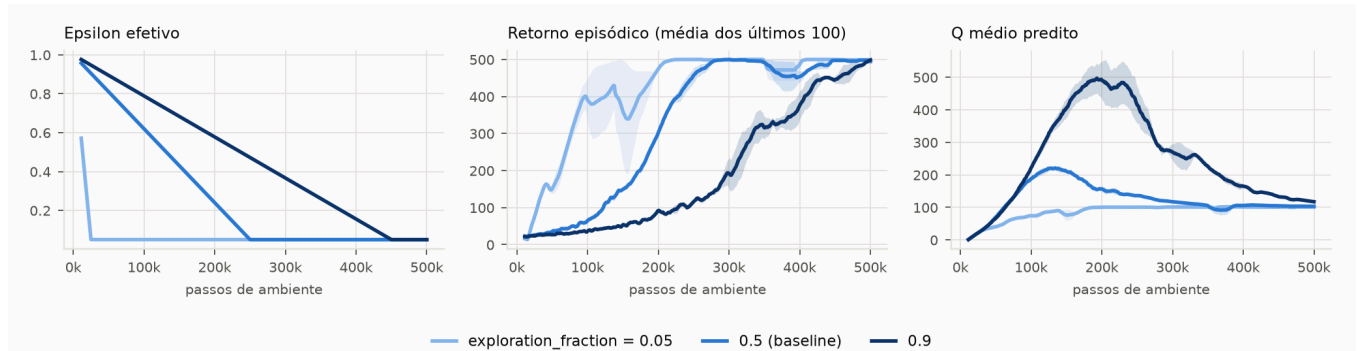
Q3 (extra) — Fração de exploração

Previsão

Com $ef=0.05$ o epsilon atinge o piso 0,05 por volta de 25k passos: aprendizado inicial rápido, com risco de estacionar numa política subótima. Com $ef=0.9$ o epsilon ainda vale $\approx 0,14$ aos 400k: esperávamos a curva de retorno de **treino** deprimida o tempo todo — ela mede a política de comportamento — mas `eval/mean_return` (greedy) bem melhor do que essa curva sugere.

Varredura, gráficos e por que estes gráficos

`dqn.exploration_fraction` $\in \{0.05, 0.5 \text{ (baseline)}, 0.9\}$, seeds 1 e 2. Escolhemos `charts/epsilon` porque é ele que mostra o cronograma **efetivamente executado** — sem ele as outras duas curvas não são interpretáveis, já que toda diferença entre elas é consequência de onde cada epsilon estava naquele passo; `episodic_return` como métrica de interesse; e `q_values` porque é o que separa “não aprendeu” de “aprendeu, a política greedy é boa, mas o comportamento ainda joga dados” — um Q correto com retorno de treino baixo só admite a segunda leitura.



Explicação

O painel de epsilon fixa as três linhas do tempo: piso em 25k, 250k e 450k passos. As três configurações terminam com avaliação greedy de 500, mas levam tempos muito diferentes para chegar lá — $ef=0.05$ cruza 450 de retorno aos **147k** passos, o baseline aos **246k** e $ef=0.9$ aos **427k**. O detalhe que fecha o mecanismo: a curva de $ef=0.9$ só sobe para 500 no intervalo 450–500k, que é **exatamente** quando seu epsilon encosta no piso. A curva de treino não estava medindo a qualidade do Q aprendido; estava medindo uma política que ainda jogava 14% de ações aleatórias, e cada ação aleatória em um episódio de 500 passos é uma chance de derrubar o bastão. Confirmação disso vem da avaliação greedy: 500 e 500 nos dois seeds de $ef=0.9$, contra um retorno de treino de 473 nos últimos 50k passos.

`q_values` acrescenta a parte não óbvia: $ef=0.9$ chega a ≈ 515 , cinco vezes o valor verdadeiro, ao passo que $ef=0.05$ se estabiliza em ≈ 101 — praticamente o valor exato de uma política que equilibra o bastão pelos 500 passos com $\gamma=0.99$. É a mesma patologia da Q2 com `buffer=500000`, e por uma razão comum: em ambos os casos o minibatch é dominado por transições geradas por uma política muito pior do que a atual, e o operador `max` sobre ações mal estimadas nessas regiões infla o alvo. Isso sugere que o hiperparâmetro relevante não é “exploração” nem “tamanho do buffer” isoladamente, e sim **quão off-policy é o conteúdo do batch**. **Previsão: confirmada na parte do mecanismo, errada na parte do risco**. A leitura “a curva de treino mede a política de comportamento, não a greedy” se sustentou exatamente como previsto; mas o risco que atribuímos a $ef=0.05$ — travar numa política subótima — não apareceu: no CartPole o piso `end_e=0.05` já garante exploração suficiente, e cortar a fração de exploração de 0,5 para 0,05 deixou o treino **1,7x mais rápido** sem custo nenhum na política final.

Avaliação greedy final (10 episódios, por seed) e retorno médio de treino nos últimos 50k passos

	tnf=1	tnf=5000	buf=128	buf=500	buf=500k	ef=0.05	ef=0.9	baseline
eval seed 1 / 2	272 / 270	176 / 290	344 / 500	500 / 500	261 / 151	500 / 500	500 / 500	485 / 500
retorno últ. 50k	231	259	435	500	274	499	473	497
q_values final	65	58	135	100	172	101	124	104

Reprodução: `python tools/run_sweep.py` (16 runs, 3,3 min em CPU num M4 Pro, 7 simultâneos) e `python tools/make_figures.py`. Implementação: `ReplayBuffer.add` (escrita circular), `ReplayBuffer.sample` (uniforme sobre `[0, size)`, nunca sobre os slots vazios) e `compute_td_targets` ($r + \gamma \max_a Q_{\text{target}(s', a)}(1 - \text{done})$, achatado para `(B, 1)`). `python -m pytest tests/test_dqn.py: 7/7`.